Bank Account Management System Using C++

Cornerstone project work submitted to Aditya University in partial fulfilment of the requirements for the award of the Degree of

B.Tech. degree

in

Artificial Intelligence and Machine Learning (2026-2027)

SUBMITTED BY

G.Srinivas	**25B11AI349**
P.Sai Tharun	**25B11AIA53**
B.Venkatesh	**25B11AI136**
T.Harika	**25B11AI982**

UNDER THE GUIDANCE OF

T. Vijaya Lakshmi Madam

Designation

Department of Artificial Intelligence & Machine Learning

ADITYA UNIVERSITY

Department of Computer Science & Engineering

Aditya University, Aditya Nagar, ADB Road,
Surampalem, Kakinada, District, Andhra Pradesh, India - 533437

DECLARATION

I hereby declare that the Cornerstone project work entitled "**BANK ACCOUNT MANAGEMENT SYSTEM**" is being submitted by me for the award of the B. Tech Degree in Computer Science and Engineering, Aditya University, Aditya Nagar, ADB Road, Surampalem is a record of original work carried out by us. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the Institutional Academic Integrity and AI Usage Policy.

Submitted By

G. Srinivas	25B11AI349
P. Sai Tharun	25B11AIA53
B. Venkatesh	25B11AI136
T. Harika	25B11AI982

Place:

Date:

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ADITYA UNIVERSITY, ADITYA NAGAR,

SURAMPALEM.

This is to certify that the cornerstone project entitled "**BANK ACCOUNT MANAGEMENT SYSTEM**" is being submitted by **G.Srinivas , P.Sai Tharun , B.Venkatesh , T.Harika** in partial fulfilment for the award of B. Tech Degree from Aditya University, is a record of original work done by them under my supervision. The contents of this project, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Project Guide

T. Vijaya Lakshmi

Assistant Professor

Department of AIML

Aditya University

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **T.Vijaya Lakshmi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Bolla Manikyala Rao, Associate Professor, Dr. Dasari Veeraswamy, Assistant Professor** Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. Kovvuri Naga Bhargavi , Associate Professor & HOD**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **DR. M.V. RAJESH, ASSOCIATE DEAN, SCHOOL OF COMPUTING**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **DR. G. SURESH, REGISTRAR; DR. A. RAMESH, PRO VICE-CHANCELLOR (ENGINEERING & SCIENCES); DR. S. RAMA SREE, PRO VICE-CHANCELLOR (ACADEMICS); DR. M.B. SRINIVAS, VICE-CHANCELLOR; AND DR. M. SREENIVASA REDDY, DEPUTY PRO CHANCELLOR AND MANAGEMENT, ADITYA UNIVERSITY**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **FACULTY MEMBERS, LAB TECHNICIANS, NON-TEACHING STAFF, AND OUR FRIENDS** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

The **Bank Account Management System** is a C++-based application developed to simulate basic banking activities in a simple, organized, and user-friendly environment. The system is designed to manage customer bank accounts and perform common banking operations such as account creation, deposit, withdrawal, balance enquiry, account search, account update, and viewing account details.

The main purpose of this project is to provide a practical implementation of **Object-Oriented Programming concepts using C++**. The system demonstrates how classes, objects, data members, and member functions can be used to organize banking-related information and operations.

The application maintains customer and account information and supports transaction-related activities. Deposit and withdrawal operations update the account balance, while transaction information can be maintained using file storage. The system also provides search and update facilities so that account information can be managed efficiently.

This project is developed for educational purposes and helps students understand the basic working of a banking system without involving actual banking transactions. It provides practical knowledge of account management, transaction handling, data organization, file storage, and OOP concepts in C++.

ABBREVATIONS

OOP – Object-Oriented Programming

C++ – C Plus Plus

CSV – Comma-Separated Values

UI – User Interface

ID – Identification

ATM – Automated Teller Machine

DB – Database

PIN – Personal Identification Number

P2P – Person-to-Person

API – Application Programming Interface

CRUD – Create, Read, Update and Delete

CLI – Command Line Interface

LIST OF FIGURES

Figure 1.1 System Architecture of Bank Account Management System

Figure 1.2 Overall System Flowchart

Figure 2.1 Login Screen

Figure 2.2 Main Menu

Figure 2.3 Account Creation Output

Figure 2.4 Deposit Operation

Figure 2.5 Withdrawal Operation

Figure 2.6 Balance Enquiry

Figure 2.7 Search Account

Figure 2.8 Update Account

Figure 2.9 Transaction Records

Figure 2.10 File Storage

LIST OF TABLES

Table 1.1 Functional Requirements of the System

Table 1.2 Planned Modules and Their Functions

Table 1.3 System Input Validation Rules

Table 2.1 OOP Concepts Used in the Project

Table 2.2 Account Data Fields

Table 2.3 Transaction Data Fields

Table 2.4 Sample Test Cases and Expected Results

TABLE OF CONTENTS

CONTENTS	PAGE NO
Title Page	I
Certificate	II
Declaration	III
Acknowledgement	IV
Abstract	V
Abbreviations	VI
List of Figures	VII
List of Tables	VIII
CHAPTER -1 **Project Planning**	
1.1. Problem statement	
1.2. Project Objective	
1.3. Planned Modules	
1.4. System Architecture (Algorithm & Flowchart)	
CHAPTER -1 **Project Implementation**	
2.1. OOP Concepts Used for implementation	
2.2. Code	
2.3. Output Screens	
CHAPTER-3 **Conclusion & Future Scope**	
3.1. Conclusion	
3.2. Future Scope	

Problem statement

Banking is an important activity that involves maintaining customer details, account information, balances, and transaction records. Managing these details manually can be time-consuming and may lead to difficulties in maintaining accurate records. Searching for a particular customer account, updating account information, recording deposits and withdrawals, and checking the current balance can also become difficult when the number of records increases.

A simple computerized system is therefore required to organize basic banking activities in an efficient and user-friendly manner. The system should provide facilities for creating customer accounts, storing account details, depositing money, withdrawing money, checking account balances, searching for accounts, and updating account information.

The **Bank Account Management System Using C++** is developed to address these requirements. The system provides a simple environment for managing basic banking operations and maintaining customer and transaction information.

The project also aims to demonstrate how **Object-Oriented Programming concepts in C++**, such as classes, objects, data members, and member functions, can be applied to a practical application.

> The proposed system is designed for educational purposes and helps users understand account management, transaction handling, and basic financial activities in a simple and organized manner.

Need for the Proposed System

The proposed Bank Account Management System provides a structured way to handle basic banking activities through a C++ application. Instead of maintaining customer information manually, the system organizes account details and transactions in a systematic manner. This makes operations such as account creation, deposit, withdrawal, balance enquiry, search, and account update easier to manage.

The system also applies input validation before important operations. For example, the withdrawal operation checks whether sufficient balance is available before changing the account balance. Account identification can be performed using account number or customer name, and invalid or unavailable account details can be handled by displaying an appropriate message to the user.

The project is intended as an educational banking application and does not perform real financial transactions. It demonstrates how Object-Oriented Programming concepts such as classes, objects, data members, member functions, and encapsulation can be applied to a real-world-inspired problem. The system can also maintain customer, account, and transaction records using simple file storage.

The main objective of the Bank Account Management System is to develop a simple and organized C++ application for managing basic banking operations.

The specific objectives of the project are:

1. To develop a simple and user-friendly Bank Account Management System using C++.
2. To provide a facility for creating new customer bank accounts.
3. To store and manage customer and account-related information.
4. To provide a deposit facility for adding money to a customer's account.
5. To provide a withdrawal facility for withdrawing available money from an account.
6. To check the available balance before processing a withdrawal.
7. To provide a balance enquiry facility for displaying the current account balance.
8. To provide a search facility for finding accounts using account number or customer name.
9. To provide an update facility for modifying customer or account details.
10. To maintain records of deposits and withdrawals through the transaction module.
11. To save account and transaction information using CSV/Text file storage.
12. To demonstrate the practical implementation of Object-Oriented Programming concepts in C++.
13. To provide students with practical knowledge of banking operations, account management, transaction handling, and basic financial activities.

The objectives are focused on creating an organized educational banking application rather than a real-world banking platform.

Planned Modules

The Bank Account Management System is divided into several modules. Each module is responsible for a specific functionality of the system.

1.3.1 Login Module

The Login Module provides access to authorized users of the system. It acts as the entry point to the application.

The module is intended to provide controlled access before users perform banking operations. After successful access, the user can proceed to the main banking functionalities.

Main purpose:

- Provide login access.
- Allow authorized users to use the system.
- Provide an initial security layer for the application.

1.3.2 Account Creation Module

The Account Creation Module is used to create a new bank account for a customer.

The module collects the required customer and account information and creates an account record.

Main functions:

- Enter customer details.
- Enter account information.
- Create a new account.
- Store the account information.
- Maintain the newly created account record.

This module forms the basic foundation of the account management system.

1.3.3 Account Details Module

The Account Details Module is used to store and display customer and account information.

It helps users view the information associated with a particular account in an organized manner.

The module can be used along with other functionalities such as account search, balance enquiry, deposit, and withdrawal.

1.3.4 Deposit Module

The Deposit Module allows money to be added to a customer's account.

When a user selects the deposit operation, the relevant account is identified and the deposit amount is entered. The amount is then added to the existing account balance and the transaction can be recorded.

Deposit process:

Account Selection
↓
Enter Deposit Amount
↓
Validate Transaction
↓
Add Amount to Balance
↓
Update Account
↓
Record Transaction

The deposit module helps demonstrate transaction processing and balance updating.

1.3.5 Withdrawal Module

The Withdrawal Module allows users to withdraw available money from a customer's account.

Before processing the withdrawal, the system checks the available balance. If sufficient balance is available, the withdrawal can be processed and the account balance can be updated.

Withdrawal process:

Account Selection
↓
Enter Withdrawal Amount
↓
Check Available Balance
↓
Process Withdrawal
↓
Update Balance
↓
Record Transaction

This module demonstrates how transaction validation can be applied before modifying account information.

1.3.6 Balance Enquiry Module

The Balance Enquiry Module displays the current balance of a selected account.

The user provides the required account information, and the system retrieves the corresponding account balance.

This functionality allows the user to quickly determine the current balance without performing any transaction.

1.3.7 Search Account Module

The Search Account Module is used to find an existing customer account.

The PPT specifies two search methods:

- Search using account number.
- Search using customer name.

The module retrieves the corresponding account information when a matching record is available.

This functionality makes it easier to locate customer accounts from the stored records.

1.3.8 Update Account Module

The Update Account Module is used to modify existing customer or account details.

When an existing account needs to be updated, the system identifies the relevant account and modifies the required information.

The updated information is then maintained in the account records.

1.3.9 Transaction Module

The Transaction Module is responsible for recording banking transactions.

The main transaction types handled by the system are:

- Deposit

- Withdrawal

The transaction module helps maintain information related to transactions performed on customer accounts.

It also supports updating the account balance after successful transactions.

1.3.10 File Storage Module

The File Storage Module is used to save account and transaction records in files.

The project presentation specifies **CSV/Text File Storage**.

The module can maintain:

- Customer records
- Account records
- Transaction records

File storage allows the system to maintain records in an organized manner and provides a simple method of storing information used by the application.

1.4 System Architecture (Algorithm & Flowchart)

1.4.1 System Architecture

The Bank Account Management System follows a modular architecture. The system starts with user access and then provides different account management and transaction functionalities.

The overall architecture can be represented as:

User

↓

Login Module

↓

Main Menu

↓

Account Management

- Account Creation
- Account Details
- Search Account
- Update Account

↓

Banking Operations

- Deposit
- Withdrawal
- Balance Enquiry

↓

Transaction Module

↓

File Storage

↓

Customer / Account / Transaction Records

The modular structure makes the system easier to understand because each operation is separated according to its purpose.

1.4.2 Algorithm

Step 1: Start the Bank Account Management System.

Step 2: Display the login interface.

Step 3: Accept the required login information.

Step 4: Provide access to the main system.

Step 5: Display the available banking operations.

Step 6: Select the required operation.

Step 7: If Account Creation is selected, enter customer and account information and create a new account.

Step 8: If Account Details is selected, retrieve and display the required account information.

Step 9: If Deposit is selected, identify the account and enter the deposit amount.

Step 10: Validate the deposit operation and update the account balance.

Step 11: Record the deposit transaction.

Step 12: If Withdrawal is selected, identify the account and enter the withdrawal amount.

Step 13: Check whether sufficient balance is available.

Step 14: If sufficient balance is available, process the withdrawal and update the account balance.

Step 15: Record the withdrawal transaction.

Step 16: If Balance Enquiry is selected, retrieve and display the current account balance.

Step 17: If Search Account is selected, search using account number or customer name.

Step 18: If Update Account is selected, modify the required customer or account information.

Step 19: Save the required customer, account, and transaction information using file storage.

Step 20: Display the result of the selected operation.

Step 21: Return to the main menu if another operation is required.

Step 22: Stop the system.

1.4.3 Flowchart

The flowchart of the system can be represented as follows:

START

↓

LOGIN

↓

Display Main Menu

↓

Select Operation

↓

Is Account Creation Selected?

→ Yes → Enter Customer Details → Create Account → Save Account

→ No → Continue

↓

Is Deposit Selected?

→ Yes → Select Account → Enter Amount → Update Balance → Record Transaction

→ No → Continue

↓

Is Withdrawal Selected?

→ Yes → Select Account → Enter Amount → Check Balance → Process Withdrawal → Update Balance → Record Transaction

→ No → Continue

↓

Is Balance Enquiry Selected?

→ Yes → Select Account → Display Balance

→ No → Continue

↓

Is Search Account Selected?

→ Yes → Search by Account Number / Customer Name → Display Details

→ No → Continue

↓

Is Update Account Selected?

→ Yes → Select Account → Modify Details → Save Updated Details

→ No → Continue

↓

Save Records

↓

Display Result

↓

Continue?

→ Yes → Main Menu

→ No → **END**

CHAPTER – 2

PROJECT IMPLEMENTATION

2.1 OOP Concepts Used for Implementation

The Bank Account Management System is implemented using **Object-Oriented Programming concepts in C++**. The project demonstrates the practical application of classes, objects, data members, and member functions.

2.1.1 Classes

A class is used to organize related data and functions into a single unit.

In the Bank Account Management System, a class can represent account-related information and operations.

A bank account class can contain information related to:

- Customer details
- Account number
- Account information
- Balance

It can also contain functions for performing operations such as:

- Account creation
- Deposit
- Withdrawal
- Balance enquiry
- Account information management

Classes help organize the program and make the implementation easier to understand.

2.1.2 Objects

Objects are created from classes and represent individual instances of the class.

In this project, individual bank accounts can be represented using objects of the appropriate class.

Each object can contain its own account information and can perform the operations defined by its class.

For example, different customer accounts can be represented as different objects while using the same class structure.

2.1.3 Data Members

Data members are variables used to store information associated with an object.

For the Bank Account Management System, data members can represent information such as:

- Customer name
- Account number
- Account details
- Account balance

These data members allow the system to maintain the information required for account management.

2.1.4 Member Functions

Member functions are functions defined inside a class that perform operations on the object's data.

In this project, member functions can be used for:

- Creating an account
- Displaying account details
- Depositing money
- Withdrawing money
- Checking account balance
- Searching account information
- Updating account information

Member functions allow the operations and data associated with an account to be organized together.

2.1.5 Encapsulation

The project can use encapsulation by keeping account-related data and the functions that operate on that data together within a class.

This provides a structured way of managing account information and reduces unnecessary access to internal data.

2.1.6 Object-Oriented Design

The use of OOP makes the project modular and easier to understand. Account-related information and operations can be grouped into suitable classes, while individual accounts can be represented through objects.

This approach demonstrates how C++ can be used to develop a practical application based on real-world entities and operations.

2.2 Code

The implementation of the project is carried out using **C++**.

The source code contains the classes, objects, data members, member functions, and other programming statements required to implement the Bank Account Management System.

The major implementation areas are:

Account Creation

The code accepts customer and account information and creates an account record.

Deposit

The code accepts a deposit amount and updates the account balance.

Withdrawal

The code checks the available balance before processing a withdrawal and updates the balance after a successful transaction.

Balance Enquiry

The code retrieves and displays the current account balance.

Search Account

The code searches stored records using account number or customer name.

Update Account

The code modifies the required customer or account details.

Transaction Recording

The code records deposit and withdrawal activities.

File Storage

The code uses CSV/Text file storage for maintaining customer, account, and transaction records.

Note: The actual C++ source code should be pasted in this section from the team's implemented program. The PPT describes the modules and their responsibilities but does not contain the complete source-code listing. Therefore, actual code should not be fabricated for the documentation.

2.3 Output Screens

The output screens section should contain screenshots of the actual program execution.

The recommended screenshots for the Bank Account Management System are:

2.3.1 Login Screen

This screen shows the login interface used to access the system.

Figure 2.1: Login Screen

[Insert actual screenshot here]

2.3.2 Main Menu

The main menu displays the available banking operations.

It can contain options such as:

- Account Creation
- Account Details
- Deposit
- Withdrawal
- Balance Enquiry
- Search Account
- Update Account
- Exit

Figure 2.2: Main Menu

[Insert actual screenshot here]

2.3.3 Account Creation Output

This screen displays the process of creating a new customer account.

The output should show the successful creation of the account after entering the required information.

Figure 2.3: Account Creation

[Insert actual screenshot here]

2.3.4 Deposit Output

This screen displays the deposit operation and the updated account balance.

Figure 2.4: Deposit Operation

[Insert actual screenshot here]

2.3.5 Withdrawal Output

This screen displays the withdrawal operation.

The system should verify the available balance before processing the transaction.

Figure 2.5: Withdrawal Operation

[Insert actual screenshot here]

2.3.6 Balance Enquiry Output

This screen displays the current balance of a selected account.

Figure 2.6: Balance Enquiry

[Insert actual screenshot here]

2.3.7 Search Account Output

This screen displays the result of searching for an account using account number or customer name.

Figure 2.7: Search Account

[Insert actual screenshot here]

2.3.8 Update Account Output

This screen displays the process of modifying existing customer or account information.

Figure 2.8: Update Account

[Insert actual screenshot here]

2.3.9 Transaction Record Output

This screen can display the stored transaction information for deposits and withdrawals.

Figure 2.9: Transaction Records

[Insert actual screenshot here]

2.3.10 File Storage Output

This section can contain screenshots showing the customer, account, or transaction records stored in CSV/Text files.

Figure 2.10: File Storage

[Insert actual screenshot here]

CHAPTER – 3

CONCLUSION & FUTURE SCOPE

3.1 Conclusion

The **Bank Account System Using C++** is a simple and organized application developed to demonstrate basic banking operations and Object-Oriented Programming concepts.

The system provides important functionalities such as account creation, customer and account information management, deposit, withdrawal, balance enquiry, account search, account update, transaction recording, and file storage.

The project demonstrates how a banking-related problem can be divided into different modules and implemented using C++. The use of classes, objects, data members, and member functions provides a practical understanding of Object-Oriented Programming.

The Deposit and Withdrawal modules demonstrate transaction processing, while the Balance Enquiry module provides access to current account balance information. Search and Update modules make account information easier to manage. The File Storage module provides a simple method of maintaining customer, account, and transaction records.

The project also provides experience in system planning, architecture design, algorithm development, flowchart preparation, module implementation, testing, debugging, documentation, and project presentation.

Overall, the project provides a practical educational environment for understanding **bank account management, transaction handling, file storage, and Object-Oriented Programming using C++**. It demonstrates how programming concepts can be applied to develop a simple real-world-inspired application.

3.2 Future Scope

The current project focuses on basic banking operations such as account creation, deposits, withdrawals, balance enquiry, account search, account updates, transaction recording, and file storage.

The system can be further developed in the future by extending its existing functionality.

Possible future improvements include:

3.2.1 Enhanced User Authentication

The login functionality can be developed further to provide different access levels for administrators, employees, and customers.

3.2.2 Improved Data Management

The existing CSV/Text File Storage can be extended to a more advanced database-based storage system for managing larger amounts of customer, account, and transaction information.

3.2.3 Transaction History

A detailed transaction-history facility can be developed to allow users to view previous deposits and withdrawals associated with an account.

3.2.4 Improved Search

The search functionality can be expanded to provide more ways of finding customer and account information.

3.2.5 Better User Interface

The system can be enhanced with a more user-friendly graphical interface instead of a basic console-based interface.

3.2.6 Additional Account Operations

More account-management operations can be added as the project is expanded, depending on the requirements of the application.

3.2.7 Improved Record Management

More advanced methods of maintaining, updating, and retrieving customer, account, and transaction records can be implemented.

3.2.8 Advanced System Features

The project can be further extended with additional banking-related functionalities while maintaining the basic OOP structure developed in the current system.

Thus, the current Bank Account Management System provides a foundation that can be expanded into a more comprehensive account-management application in future versions.